Distilling the Knowledge in a Neural Network

Hinton, Vinyals and Dean (2015), arXiv:1503.02531

Paper summary for the CS-781 capstone

The paper at a glance

Citation

G. Hinton, O. Vinyals, J. Dean. *Distilling the Knowledge in a Neural Network*. NIPS Deep Learning Workshop, 2014 (arXiv, March 2015). Google.

Why it matters for our capstone

A knowledge-distillation experiment is **mandatory** in Phases 2–7. Our Phase-1 code already implements the loss from this paper.

One-sentence summary

Train a small "student" model to match the *softened* output probabilities of a large "teacher", so the student inherits how the teacher generalises.

Where it appears in our slides

Main deck: "Compression: knowledge distillation". Glossary: "Knowledge distillation" section.

Roadmap

1. The problem
2. The basic idea
3. Experiments
4. Assessment and outlook

Training and deployment want different models

- **Training** can be slow and use huge compute. Its job is to extract structure from large, redundant data.
- **Deployment** must be fast and cheap for many users.
- The best accuracy often comes from *cumbersome* models: an ensemble of many nets, or one very large net with strong regularisation such as dropout.
- These are too expensive to serve.

Question: can a small model keep the cumbersome model's accuracy?

The paper's analogy

Many insects have a **larva** built to feed and grow, and an **adult** built to travel and reproduce. Likewise: train a big "larval" model to learn, then transform the knowledge into a small "adult" model for deployment.

Prior work

Buciluǎ, Caruana et al. (2006) showed that an ensemble's knowledge can be compressed into a single model. This paper generalises their method.

What is “knowledge”?

Usual view: knowledge = the learned parameter values. Then it is hard to see how a *different* model could hold the same knowledge.

Paper's view: knowledge = the learned *mapping from inputs to outputs*. Any model that reproduces the mapping has the knowledge.

A trained classifier assigns small probabilities to all the *wrong* classes, and **some are much larger than others**. These ratios show how the model generalises.

The paper's example

A photo of a BMW:

- P(BMW) is high.
- P(garbage truck) is tiny, but...
- P(carrot) is *much* tinier still.

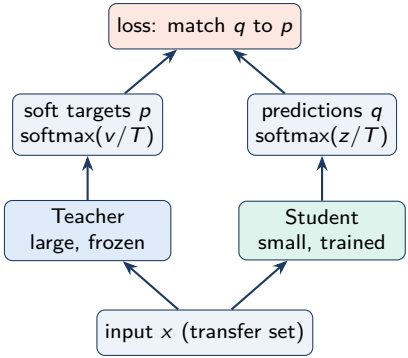
“A BMW looks more like a truck than a carrot” is real knowledge. The one-hot label (BMW = 1, others = 0) throws it away.

Soft targets

- **Hard target**: the one-hot true label.
- **Soft target**: the teacher's full probability distribution over classes.
- Train the student to match the soft targets on a **transfer set**. This can be the original training set, or even unlabelled data.

Why soft targets help:

- High-entropy soft targets carry **more information per example**.
- Their gradients have **less variance** between examples.
- So the student can learn from *less data*, with a *higher learning rate*.



The problem with confident teachers, and the fix: temperature

On easy data the teacher is almost always right and very confident. The useful information hides in *ratios of tiny probabilities*.

Paper's MNIST example: one "2" gets 10^{-6} for "3" and 10^{-9} for "7"; another "2" is the other way round. This is rich similarity structure, but it barely affects cross-entropy because the numbers are near 0.

Fix: raise the softmax **temperature** T:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Higher T gives a softer distribution. Use the same high T for teacher and student during training, then $T = 1$ at deployment.

Easy example: legal next word

Teacher logits after "The appeal is hereby": dismissed 6, rejected 4, allowed 1.

	dism.	rej.	allowed
$T = 1$	0.876	0.118	0.006
$T = 4$	0.528	0.320	0.151

At $T = 1$ the student mostly learns "dismissed". At $T = 4$ it also learns that "rejected" is a close substitute and "allowed" is not far off. That is the dark knowledge.

The distillation objective

When true labels are available, use a **weighted average of two losses**:

$$\mathcal{L} = \underbrace{T^2 \cdot \mathrm{CE}\big(p^{(T)},\, q^{(T)}\big)}_{\text{match teacher at temperature } T} + \lambda \underbrace{\mathrm{CE}\big(y,\, q^{(1)}\big)}_{\text{true label at } T=1}$$

- Both terms use the *same* student logits.
- The paper found best results with a **considerably lower weight** on the hard-label term.
- **Why** T^2: soft-target gradients shrink as $1/T^2$. Multiplying by T^2 keeps the balance of the two terms fixed when you change T.

Our capstone implementation

`legal_lm/trainer.py` uses the equivalent KL form:

$$\alpha\, \mathcal{L}_{\mathrm{CE}} + (1-\alpha)\, T^2\, \mathrm{KL}\big(p^{(T)} \,\|\, q^{(T)}\big)$$

with $\alpha = 0.5$ and $T = 2$ by default. KL and cross-entropy differ only by the teacher's entropy, a constant for the student.

Intuition

"Err toward the correct answer": the student usually cannot match the teacher exactly, and a little pull toward the true label helps.

Matching logits is a special case

Gradient of the soft-target loss for student logit z_i (teacher logits v_i):

$$\frac{\partial C}{\partial z_i} = \frac{1}{T}(q_i - p_i)$$

For high T, $e^{x/T} \approx 1 + x/T$. If the logits are zero-mean per example:

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{NT^2}(z_i - v_i)$$

So at **high temperature**, distillation = minimising $\frac{1}{2}(z_i - v_i)^2$, which is exactly Caruana's logit matching.

What temperature does

- **High** T: match all logits, including very negative ones.
- **Lower** T: pay little attention to very negative logits. These are almost unconstrained by the teacher's training, so they may be noise, or may carry useful knowledge.

Which effect wins is empirical. For *very small* students, intermediate temperatures worked best.

MNIST: distillation transfers generalisation

Model	Test errors
Teacher: 2×1200 units, dropout, jitter	67
Student: 2×800 units, no regularisation	146
Same student + soft targets at $T = 20$	**74**

The student learned the teacher's robustness to shifted images, even though the transfer set contained *no* shifted images.

Temperature: with 300 or more units per layer, all $T > 8$ worked similarly. With only 30 units, T between 2.5 and 4 was clearly best.

The "mythical digit" experiment

Remove every "3" from the transfer set. The student never sees a 3.

- Result: 206 errors, 133 of them on the 1,010 test 3s.
- Most errors come from a too-low bias for class 3. Raising that bias by 3.5 gives 109 errors, only 14 on 3s.
- So the student gets **98.6% of 3s right** without ever seeing one.

The soft targets for other digits ("this 8 looks a bit like a 3") taught it what a 3 is.

Speech recognition: an ensemble in one model

Android voice-search acoustic model: 8 hidden layers of 2,560 ReLUs, 14,000 output states, about 85M parameters. Trained on about 2,000 hours of speech (700M examples).

System	Frame acc.	WER
Baseline	58.9%	10.9%
10× ensemble	61.1%	10.7%
Distilled single model	**60.8%**	**10.7%**

Temperatures tried: 1, 2, 5, 10; weight 0.5 on the hard-label loss.

Reading the result

The ensemble improves frame accuracy by 2.2 points. The distilled model keeps 1.9 of them, **more than 80%**, at one-tenth of the inference cost. The word-error-rate gain is fully kept.

Compare

An earlier method distilled at $T = 1$ and closed only 28% of the gap between the small and large models.

Soft targets as a regulariser

Train the 85M speech model on only **3% of the data** (about 20M examples):

System	Train acc.	Test acc.
Baseline, 100% data	63.4%	58.9%
Baseline, 3% data	67.3%	44.5%
Soft targets, 3% data	65.4%	**57.0%**

With hard labels, 3% of the data overfits badly and needed early stopping. With soft targets, the model simply converged to 57.0%, within 2 points of using all the data.

Why this matters for small data

A soft target such as "60% dismissed, 30% rejected" says much more than the label "dismissed". Each example teaches more, so fewer examples are needed.

Our Phase-2 NER data has only two annotated folds, a small-data setting where this effect should help.

Specialist models for very large datasets

Setting: Google's JFT, with 100M images and 15,000 classes. The baseline network took about six months to train, so a full ensemble was impossible.

Idea: one *generalist* plus many *specialists*. Each specialist:

- focuses on a cluster of **easily confused** classes, found by K-means on the covariance of the generalist's predictions;
- starts from the generalist's weights, and trains on half special-class and half random data;
- lumps all other classes into one "dustbin" class.

At test time only specialists relevant to the generalist's top guess run; outputs are combined by minimising a sum of KL divergences.

Example clusters from the paper

- Tea party; Easter; Bridal shower; Baby shower...
- Bridge; Cable-stayed bridge; Suspension bridge; Viaduct...
- Toyota Corolla E100; Opel Signum; Opel Astra...

Results

61 specialists with 300 classes each. Test accuracy rose from 25.0% to 26.1%, a 4.4% relative gain. Gains grew with the number of specialists covering a class, up to about 16%. Specialists trained in days, not weeks, fully in parallel.

Specialists vs mixtures of experts

Mixture of experts (Jacobs et al., 1991)

A gating network learns which expert handles each example, while the experts learn.

- Assignments use each expert's actual performance, which is good.
- But every expert's data depends on all the others, and the gate must compare experts on the same example.
- So training is **hard to parallelise**.

Specialists (this paper)

- Train a generalist first, then fix the class subsets once from its confusions.
- Each specialist trains **fully independently**.
- At test time the generalist decides which specialists to run.

Suggested next step: give specialists a full softmax and train them with the generalist's soft targets for the non-special classes, so they do not forget or overfit.

Using distillation in our capstone

Phase 1 (optional compression): `configs/distill_efficient_small.yaml` trains a 47M A4 student from an A1 teacher, with $\alpha = 0.5$ and $T = 2$.

Phases 2–7 (mandatory): teacher = a fine-tuned larger model; student = a smaller model. Examples:

- **NER**: per-token label distributions.
- **RR / CJPE / LSI**: class or multi-label probabilities.
- **PCR**: teacher similarity scores.
- **Summarisation**: per-token soft targets, or teacher-generated summaries.

Experiments worth running

- Student with distillation vs the same student trained from labels only.
- Temperature sweep, e.g. $T \in \{1, 2, 4, 8\}$. The paper predicts small students prefer intermediate T.
- Hard-label weight sweep.
- Report parameters, memory, speed and task F1 for teacher and student.

Strengths and limitations

Strengths

- Very simple: one extra loss term and one hyperparameter, T.
- Architecture-agnostic: teacher and student can differ completely.
- Works with unlabelled transfer data.
- Strong regularisation effect on small data.
- Unifies earlier logit matching as the high-T limit.

Limitations

- Only the *output layer* is transferred. Intermediate representations are ignored.
- Requires running the teacher on every training example, which is costly.
- T and the loss weight must be tuned; the best T depends on student size.
- Experiments are on classification only: MNIST, speech frames and images. Sequence generation is not studied.
- The authors had not yet distilled the specialists back into one net.

Future research directions (1): what the field did next

Direction	Representative later work
Distil intermediate layers, not just outputs	FitNets (Romero et al., 2015) match hidden "hints"; TinyBERT (Jiao et al., 2020) matches attention maps and hidden states
Distil sequence models	Sequence-level KD (Kim and Rush, 2016) trains the student on the teacher's beam-search outputs
Compress pretrained language models	DistilBERT (Sanh et al., 2019): 40% smaller and 60% faster, keeping 97% of BERT's language-understanding performance
Student as big as the teacher	Born-Again Networks (Furlanello et al., 2018): a same-size student can *outperform* its teacher
Better divergences for generation	MiniLLM (Gu et al., 2023) uses reverse KL so the student does not spread probability over bad outputs

Future research directions (2): open questions

Open in general

- **Why** does distillation work so well? Dark knowledge, label smoothing and optimisation ease are all partial explanations.
- How large can the teacher–student gap be before distillation stops helping?
- Distilling from many specialists back into one model, left open by this paper.
- Choosing temperature automatically, or per example.
- Distillation without the original data (data-free KD).

Open for legal NLP (our setting)

- Does distillation preserve *rare* but important classes, such as rare statutes in LSI, or does it favour the teacher's majority?
- Can a cross-architecture teacher (encoder–decoder) help a decoder-only student?
- Can confidential data stay in-house by distilling a big in-house model into a laptop-sized one?
- Distilling long-document reasoning, e.g. judgment prediction, into short-context students.

Summary

- Knowledge is the **input–output mapping**, not the parameters, so it can move into a different model.
- Train the student on the teacher's **soft targets**, softened with a **temperature** T, plus a small weight on the true labels. Scale the soft loss by T^2.
- Matching logits is the high-temperature special case.
- Results: a small MNIST net went from 146 to 74 errors. A single speech model kept over 80% of a 10-model ensemble's gain. With soft targets, 3% of the data reached 57.0% vs 58.9% with all of it.
- Specialists on confusable class clusters improve very large classifiers cheaply and in parallel.
- For us: the basis of the mandatory distillation experiment in Phases 2–7.

Reference

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.

Later work cited on the future-directions slide: Romero et al. (2015) FitNets; Kim and Rush (2016) Sequence-Level Knowledge Distillation; Furlanello et al. (2018) Born-Again Neural Networks; Sanh et al. (2019) DistilBERT; Jiao et al. (2020) TinyBERT; Gu et al. (2023) MiniLLM.